

period_extraction_DR_general

May 13, 2026

1 DR Cocycle Period Extraction — General Function for Weight- k Modular Forms

This notebook implements the **Diamantis–Rolen finite-endpoint cocycle** as a single, q -series-agnostic function `extract_periods(coeffs_dict, tau0, k)` that extracts the periods $\omega^\pm$ (or quasi-periods $\eta^\pm$) of any weight- k modular form on $\mathrm{SL}_2(\mathbb{Z})$, holomorphic or weakly holomorphic, from its Laurent-series coefficient dictionary.

Roadmap 1. Build exact q -series for Δ and $\widehat{\Delta}$ via sympy rationals. 2. Define the core DR machinery (integrator, slash operators, coboundary solver). 3. Package everything into `extract_periods`. 4. Test on $\Delta \rightarrow$ reproduce Brown’s $\omega^\pm$. 5. Test on $\widehat{\Delta} \rightarrow$ reproduce Brown’s $\eta^\pm$. 6. Basepoint independence check. 7. Analytic formula: exhibit the connection to L -values for the holomorphic case.

```
[1]: from mpmath import mp, mpc, mpf, pi, exp, quad, matrix, lu_solve, nstr
from sympy import Rational
from math import comb
import time

mp.dps = 45                                # 45 decimal digits throughout
TWOPII = mpc(0, 2) * pi                    # 2 i

# Helper: format an mpmath scalar with a Python format spec via float.
# Use only for error/spread diagnostics, not for the values themselves.
def _fe(x, fmt='3e'):
    return format(float(abs(x)), fmt)

print(f'Precision : {mp.dps} decimal digits')
print(f'2 i      = {TWOPII}')
```

```
Precision : 45 decimal digits
2 i      = (0.0 + 6.2831853071795864769252867665590057683943388j)
```

1.1 1. Exact q -series via sympy rational arithmetic

We build Δ , $J = j - 744$, and $\widehat{\Delta} = \Delta(J^2 + 24J - 393444)$ as dictionaries `{n: Rational}` representing Laurent series $\sum_n c_n q^n$. All arithmetic here is exact.

```

[2]: PREC_Q = 70    # q-terms through  $q^{\text{PREC\_Q}}$ 

# ---- Laurent-series helpers -----

def laurent_mul(a, b, lo, hi):
    out = {}
    for ka, va in a.items():
        for kb, vb in b.items():
            k = ka + kb
            if lo <= k <= hi:
                out[k] = out.get(k, Rational(0)) + va * vb
    return {k: v for k, v in out.items() if v != 0}

def laurent_div(num, den, lo_exp, hi_exp):
    den_min = min(den.keys())
    den_lead = den[den_min]
    quot = {}
    for target in range(lo_exp + den_min, hi_exp + den_min + 1):
        s = num.get(target, Rational(0))
        for kq in range(lo_exp, target - den_min):
            s -= quot.get(kq, Rational(0)) * den.get(target - kq, Rational(0))
        if s != 0:
            quot[target - den_min] = s / den_lead
    return quot

def divisor_sigma(n, k):
    s = 0; d = 1
    while d * d <= n:
        if n % d == 0:
            s += d**k
            if d != n // d: s += (n // d)**k
        d += 1
    return s

def eta24_qexp(prec):
    series = {0: Rational(1)}
    for n in range(1, prec + 1):
        factor = {}
        for k in range(25):
            te = n * k
            if te > prec: break
            factor[te] = Rational((-1)**k * comb(24, k))
        series = laurent_mul(series, factor, 0, prec)
    return series

# ---- Build the series -----

```

```

print('Building  $\Delta$  ...')
t0 = time.time()
eta24 = eta24_qexp(PREC_Q + 2)
Delta_series = {k + 1: v for k, v in eta24.items() if 1 <= k + 1 <= PREC_Q + 2}
print(f' first terms: {dict(list(Delta_series.items())[:5])}')
print(f' done in {time.time()-t0:.1f}s')

print('Building j, J, J2 ...')
E4 = {0: Rational(1)}
for n in range(1, PREC_Q + 3):
    E4[n] = Rational(240 * divisor_sigma(n, 3))
E4c = laurent_mul(laurent_mul(E4, E4, 0, PREC_Q + 2), E4, 0, PREC_Q + 2)
j_ser = laurent_div(E4c, Delta_series, -1, PREC_Q)
J_ser = dict(j_ser)
J_ser[0] = J_ser.get(0, Rational(0)) - Rational(744)
J_ser = {k: v for k, v in J_ser.items() if v != 0}
J_sq = laurent_mul(J_ser, J_ser, -2, PREC_Q)
print(f' J first terms: {dict(list(J_ser.items())[:4])}')

print('Building  $\Delta_{\text{Hat}} = \Delta(J^2+24J-393444)$  ...')
D_Jsq = laurent_mul(Delta_series, J_sq, -2, PREC_Q)
D_J = laurent_mul(Delta_series, J_ser, -1, PREC_Q)
a1 = -D_Jsq[0] / D_J[0]
a0 = -(D_Jsq[1] + a1 * D_J[1])
assert a1 == 24 and a0 == -393444, f'unexpected: a1={a1}, a0={a0}'
print(f' a1={a1}, a0={a0} ')

DH = {}
for k, v in D_Jsq.items(): DH[k] = DH.get(k, Rational(0)) + v
for k, v in D_J.items(): DH[k] = DH.get(k, Rational(0)) + a1 * v
for k, v in Delta_series.items(): DH[k] = DH.get(k, Rational(0)) + a0 * v
DeltaHat_series = {k: v for k, v in DH.items() if v != 0}

print(f'  $\Delta_{\text{Hat}}$  first terms: {dict(list(sorted(DeltaHat_series.items()))[:6])}')
print(' Brown: q-1=1, q2=47709536, q3=39862705122, q4=7552626810624 ')

```

Building Δ ...

first terms: {1: 1, 72: -9600560640, 71: 9791485272, 70: 1940964480, 69: 4698104544}

done in 0.0s

Building j, J, J² ...

J first terms: {-1: 1, 1: 196884, 2: 21493760, 3: 864299970}

Building $\Delta_{\text{Hat}} = \Delta(J^2+24J-393444)$...

a1=24, a0=-393444

Δ_{Hat} first terms: {-1: 1, 2: 47709536, 3: 39862705122, 4: 7552626810624, 5: 609136463852480, 6: 27947672851540608}

Brown: q⁻¹=1, q²=47709536, q³=39862705122, q⁴=7552626810624

1.2 2. Core DR cocycle machinery

All routines are weight- k agnostic (parametrised by $n = k - 2$). They are blind to whether f is holomorphic or weakly holomorphic.

```
[3]: def make_evaluator(coeffs_dict):
    """
    Build  $f : \text{mpc} \rightarrow \text{mpc}$  from a Laurent-series coefficient dict  $\{n: c_n\}$ .
     $f() = \sum_n c_n \cdot \exp(2i n)$ .
    Works for any finite contour in  $\mathbb{C}$ , holomorphic or weakly holomorphic.
    """
    terms = sorted(
        [(int(n), mpf(str(c))) for n, c in coeffs_dict.items() if c != 0]
    )
    def f(tau):
        tau = mpc(tau)
        val = mpc(0)
        for n, c in terms:
            val += c * exp(n * TWOPII * tau)
        return val
    return f

def cocycle_poly(f_eval, tau_a, tau_b, n):
    """
    Return the coefficient list  $[c_0, \dots, c_n]$  of
     $(2i)^{n+1} \int_{\tau_a}^{\tau_b} f(X - Y) dX$ ,
    where  $c_j = \text{coeff of } X^{n-j} Y^j$ .
    Integration uses mpmath GL quadrature along the straight-line segment.
    """
    tau_a, tau_b = mpc(tau_a), mpc(tau_b)
    dtau = tau_b - tau_a
    PREF = TWOPII ** (n + 1)
    coeffs = []
    for j in range(n + 1):
        def integrand(s, j=j):
            tau = tau_a + s * dtau
            return f_eval(tau) * tau**j * dtau
        moment_j = quad(integrand, [0, 1])
        coeffs.append(PREF * mpf((-1)**j) * mpf(comb(n, j)) * moment_j)
    return coeffs

def slash_T(p):
    """ $p|_T(X, Y) = p(X+Y, Y)$ .  $p[j] = \text{coeff of } X^{n-j} Y^j$ ."""
    n = len(p) - 1
    out = [mpc(0)] * (n + 1)
    for j in range(n + 1):
```

```

        for i in range(n - j + 1):
            out[j + i] += p[j] * mpf(comb(n - j, i))
    return out

def slash_S(p):
    """ $p/_S(X, Y) = p(Y, -X)$ .  $(p/_S)[m] = (-1)^m p[n-m]$ . """
    n = len(p) - 1
    return [p[n - m] * mpf((-1)**m) for m in range(n + 1)]

def solve_coboundary(C_T, n):
    """
    Solve  $P/_T - P = C_T$  for  $P = \sum_j p_j X^{n-j} Y^j$ .
    Upper-triangular system for  $m=1, \dots, n$ :
     $\sum_{j < m} p_j C(n-j, m-j) = [C_T]_m$ .
     $[C_T]_0 = 0$  is the cuspidality condition (checked externally).
     $p_n$  set to 0 ( $Y^n$  is  $T$ -fixed; choice only affects the  $X^n/Y^n$  junk
    ↪ direction).
    """
    A = matrix(n, n)
    b = matrix(n, 1)
    for m in range(1, n + 1):
        for j in range(m):
            A[m-1, j] = mpf(comb(n - j, m - j))
        b[m-1, 0] = C_T[m]
    psol = lu_solve(A, b)
    return [psol[j, 0] for j in range(n)] + [mpc(0)]

```

```

[4]: def brown_basis_k12():
    """
    Brown's cuspidal basis vectors at  $S$  for  $k=12$  ( $n=10$ ),
    from arXiv:1710.07912 §8.2.
     $P_+_S = (36/691)(Y^{10} - X^{10}) + X^2 Y^2 (X^2 - Y^2)^3$ 
     $P_-_S = 4X^9 Y - 25X^7 Y^3 + 42X^5 Y^5 - 25X^3 Y^7 + 4XY^9$ 
    Returns  $(P_+, P_-)$  as lists where index  $j$  = coeff of  $X^{10-j} Y^j$ .
    """
    P_plus = [mpf(0)] * 11
    P_minus = [mpf(0)] * 11
    P_plus[0] = -mpf(36)/mpf(691); P_plus[10] = mpf(36)/mpf(691)
    P_plus[2] = mpf(1); P_plus[4] = -mpf(3)
    P_plus[6] = mpf(3); P_plus[8] = -mpf(1)
    P_minus[1] = mpf(4); P_minus[3] = -mpf(25)
    P_minus[5] = mpf(42); P_minus[7] = -mpf(25); P_minus[9] = mpf(4)
    return P_plus, P_minus

```

```

def read_periods(C_prime_S, n=10, verbose=True):
    """
    Project  $C'_S = + P_+ S + - P_- S + c(X^n - Y^n)$  onto  $(+, -)$ .

    Interior odd monomials  $j=1,3,\dots,n-1 \rightarrow -$  (all give the same value)
    Interior even monomials  $j=2,4,\dots,n-2 \rightarrow +$  (all give the same value)
     $j=0$  and  $j=n$  carry the free coboundary ambiguity and are skipped.

    The spread across monomials is the precision diagnostic.
    """
    assert n == 10, 'Brown basis implemented only for n=10 (k=12).'
    P_plus, P_minus = brown_basis_k12()
    odd_js = list(range(1, n, 2)) # 1,3,5,7,9
    even_js = list(range(2, n, 2)) # 2,4,6,8

    minus_est = [C_prime_S[j] / P_minus[j] for j in odd_js]
    plus_est = [C_prime_S[j] / P_plus[j] for j in even_js]

    if verbose:
        print('    estimates (odd interior monomials):')
        for j, e in zip(odd_js, minus_est):
            print(f'    j={j} X^{n-j}Y^{j}: {e}')
        print('    estimates (even interior monomials):')
        for j, e in zip(even_js, plus_est):
            print(f'    j={j} X^{n-j}Y^{j}: {e}')
        spread_minus = max(abs(a - b) for a in minus_est for b in minus_est)
        spread_plus = max(abs(a - b) for a in plus_est for b in plus_est)
        print(f'    internal spread : {_fe(spread_minus)}')
        print(f'    internal spread : {_fe(spread_plus)}')

    return plus_est[0], minus_est[0]

```

1.3 3. Main function: extract_periods

Single entry point. Takes only the q -series coefficients (plus optional τ_0, k) and returns (ω^+, ω^-) . Zero knowledge of whether f is holomorphic or weakly holomorphic — the algorithm is identical.

Default $\gamma = S$ (the generator of non-trivial cohomology for $SL_2(\mathbb{Z})$). For $k \neq 12$ the Brown basis must be supplied externally (natural extension point).

```

[5]: def extract_periods(
    coeffs_dict,
    tau0      = mpc('0.3', '1.2'),
    k         = 12,
    label     = 'f',
    verbose   = True,
):
    """

```

Extract periods (+, -) - or quasi-periods (+, -) for weakly holomorphic f - via the Diamantis-Rolen finite-endpoint cocycle.

Parameters

`coeffs_dict` : dict {int: Rational/float/int}

$f() = \sum_n c_n e^{2\pi i n \tau}$. Negative-index keys (cusp poles) allowed.

`tau0` : mpc

Base point in $\mathbb{H}$ (default $0.3+1.2i$, $|q| \approx 10^{-3}$ on all paths).

`k` : int

Modular weight (default 12); polynomial degree $n = k-2$.

`label, verbose` : str, bool

Cosmetic.

Returns

`omega_plus, omega_minus` : mpc

$\text{Re}(\omega_{\text{plus}}) + i, \text{Im}(\omega_{\text{minus}}) - i$ (noise in the other part).

Steps (Diamantis-Rolen §2.1 / Brown Def 2.4)

1. `f_eval` ← `make_evaluator(coeffs_dict)`

2. $C_S \leftarrow (2i)^{k-1} \int_{-1/\tau_0}^{\tau_0} f(\tau) (X-Y)^{k-2} d\tau$

3. $C_T \leftarrow (2i)^{k-1} \int_{-1/\tau_0}^{\tau_0} f(\tau) (X-Y)^{k-2} d\tau$

4. Solve $P|_T - P = C_T$ for coboundary polynomial P .

5. $C'_S \leftarrow C_S - (P|_S - P)$

6. Project C'_S onto (+, -) via interior-monomial extraction.

"""

`n` = `k` - 2

`f_eval` = `make_evaluator(coeffs_dict)`

if `verbose`:

`sep` = ' ' * 65

`print(f'\n{sep}')`

`print(f' extract_periods({label}) k={k}, n={n}')`

`print(f' tau0 = {tau0}')`

`print(sep)`

`t0` = `time.time()`

`C_S` = `cocycle_poly(f_eval, -1/tau0, tau0, n)`

if `verbose`: `print(f' C_S computed ({time.time()-t0:.1f}s)')`

`t0` = `time.time()`

`C_T` = `cocycle_poly(f_eval, tau0 - 1, tau0, n)`

if `verbose`:

`print(f' C_T computed ({time.time()-t0:.1f}s)')`

`print(f' Cuspidality |[C_T]_{X^n}| = {fe(abs(C_T[0]))} (→ 0)')`

```

p          = solve_coboundary(C_T, n)
PS         = slash_S(p)
C_prime_S = [C_S[j] - (PS[j] - p[j]) for j in range(n + 1)]

if verbose: print()
return read_periods(C_prime_S, n=n, verbose=verbose)

```

1.4 4. Validation: Δ (classical discriminant)

Target (Brown 1710.07912 §8.3):

$$\omega^+ = -68916772.809595194754 \dots, \quad \omega^-/i = -5585015.3793104018668 \dots$$

```

[6]: TAU0 = mpc('0.3', '1.2')

omega_plus, omega_minus = extract_periods(
    dict(Delta_series), tau0=TAU0, k=12, label='Δ'
)

print()
print(' ' * 65)
print('RESULTS Δ')
print(' ' * 65)

OP_target = mpf('-68916772.809595194754')
OM_target = mpf('-5585015.3793104018668')

print('      =', omega_plus)
print(' /i =', omega_minus / mpc(0, 1))
print()
print(f' Brown      = {OP_target}...')
print(f' Brown /i = {OM_target}...')

rel_plus  = abs(omega_plus      - OP_target) / abs(OP_target)
rel_minus = abs(omega_minus/mpc(0,1) - OM_target) / abs(OM_target)
print()
print(f' |Δ | / | | = {_fe(rel_plus)}')
print(f' |Δ | / | | = {_fe(rel_minus)}')

```

```

extract_periods(Δ)    k=12, n=10
    = (0.3 + 1.2j)

```

C_S computed (2.6s)

C_T computed (2.6s)
 Cuspidality $|[C_T]_{\{X^{10}\}}| = 2.395204e-42 \quad (\rightarrow 0)$

```

estimates (odd interior monomials):
j=1  X^9Y^1: (3.67341984631964846240230167881951774318332986e-40 -
5585015.37931040186687713926379627512963503343j)
j=3  X^7Y^3: (9.40395480657830006374989229777796542254932445e-40 -
5585015.37931040186687713926379627512963503343j)
j=5  X^5Y^5: (2.79879607338639882849699175529106113766348942e-40 -
5585015.37931040186687713926379627512963503343j)
j=7  X^3Y^7: (9.40395480657830006374989229777796542254932445e-40 -
5585015.37931040186687713926379627512963503343j)
j=9  X^1Y^9: (3.67341984631964846240230167881951774318332986e-40 -
5585015.37931040186687713926379627512963503343j)
estimates (even interior monomials):
j=2  X^8Y^2: (-68916772.8095951947543101246553310304390699691 -
8.81620763116715630976552402916684258363999168e-39j)
j=4  X^6Y^4: (-68916772.8095951947543101246553310304390699691 -
3.91831450274095835989578845740748559272888519e-39j)
j=6  X^4Y^6: (-68916772.8095951947543101246553310304390699691 -
7.83662900548191671979157691481497118545777038e-39j)
j=8  X^2Y^8: (-68916772.8095951947543101246553310304390699691 -
8.81620763116715630976552402916684258363999168e-39j)
internal spread : 9.317461e-40
internal spread : 1.273452e-38

```

RESULTS Δ

```

= (-68916772.8095951947543101246553310304390699691 -
8.81620763116715630976552402916684258363999168e-39j)
/i = (-5585015.37931040186687713926379627512963503343 -
3.67341984631964846240230167881951774318332986e-40j)

```

```

Brown      = -68916772.809595194754...
Brown /i    = -5585015.3793104018668...

```

```

|Δ | / | | = 4.499988e-21
|Δ | / | | = 1.381183e-20

```

1.5 5. Validation: $\widehat{\Delta}$ (weakly holomorphic)

Target (Brown 1710.07912 §8.3 / `delta_and_hatdelta_periods_executed.ipynb`):

$$\eta^+ = 127202100647.17709477 \dots, \quad \eta^-/i = 10276732343.649132750 \dots$$

`extract_periods` receives the Δ Hat q-series (including the q^{-1} cusp-pole term) and runs **identically the same code** as for Δ . No modification whatsoever.

```

[7]: eta_plus, eta_minus = extract_periods(
    dict(DeltaHat_series), tau0=TAU0, k=12, label='ΔHat'
)

print()
print(' ' * 65)
print('RESULTS ΔHat')
print(' ' * 65)

EP_target = mpf('127202100647.1770947773171612986')
EM_target = mpf('10276732343.64913275081719307240')

print('      =', eta_plus)
print('    /i =', eta_minus / mpc(0, 1))
print()
print(f' ref      = {EP_target}...')
print(f' ref    /i = {EM_target}...')

rel_eplus = abs(eta_plus - EP_target) / abs(EP_target)
rel_eminus = abs(eta_minus/mpc(0,1) - EM_target) / abs(EM_target)
print()
print(f' |Δ | / | | = {_fe(rel_eplus)}')
print(f' |Δ | / | | = {_fe(rel_eminus)}')

```

```

extract_periods(ΔHat)    k=12, n=10
    = (0.3 + 1.2j)

```

C_S computed (2.9s)

C_T computed (4.9s)

Cuspidality $|[C_T]_{\{X^{10}\}}| = 8.527593e-36 \rightarrow 0$

estimates (odd interior monomials):

j=1 X^9Y^1 : $(-8.66668474974256133875190074163217293342145742e-34 + 10276732343.6491327508171930724009209088993992j)$

j=3 X^7Y^3 : $(-1.47911419728939713514699105990522418063726207e-33 + 10276732343.6491327508171930724009209088993992j)$

j=5 X^5Y^5 : $(-1.87090337454760054296866428708250082372272732e-33 + 10276732343.6491327508171930724009209088993995j)$

j=7 X^3Y^7 : $(-1.41748443906900558784919976574250650644404281e-33 + 10276732343.6491327508171930724009209088993992j)$

j=9 X^1Y^9 : $(-5.77778983316170755916793382775478195561430494e-34 + 10276732343.6491327508171930724009209088993993j)$

estimates (even interior monomials):

j=2 X^8Y^2 : $(127202100647.177094777317161298610877494045995 + 1.23259516440783094595582588325435348386438505e-32j)$

```

j=4 X^6Y^4: (127202100647.177094777317161298610877494045981 +
2.25975780141435673425234745263298138708470593e-32j)
j=6 X^4Y^6: (127202100647.177094777317161298610877494045986 +
2.20839966956403044483752137416404999192368989e-32j)
j=8 X^2Y^8: (127202100647.177094777317161298610877494045995 +
9.24446373305873209466869412440765112898288791e-33j)
internal spread : 1.311651e-33
internal spread : 1.888132e-32

```

RESULTS ΔHat

```

= (127202100647.177094777317161298610877494045995 +
1.23259516440783094595582588325435348386438505e-32j)
/i = (10276732343.6491327508171930724009209088993992 +
8.66668474974256133875190074163217293342145742e-34j)

ref      = 127202100647.1770947773171612986...
ref /i    = 10276732343.6491327508171930724...

|Δ | / | | = 8.551348e-32
|Δ | / | | = 8.961106e-32

```

1.6 6. Basepoint independence

Cohomological invariance requires the result to be independent of τ_0 . We re-run both at $\tau_0 = -0.2 + 0.9i$ and report the differences.

```

[8]: TAU0_ALT = mpc('-0.2', '0.9')

op_alt, om_alt = extract_periods(
    dict(Delta_series), tau0=TAU0_ALT, k=12, label='Δ (alt )',
    verbose=False
)
ep_alt, em_alt = extract_periods(
    dict(DeltaHat_series), tau0=TAU0_ALT, k=12, label='ΔHat (alt )',
    verbose=False
)

print('BASEPOINT INDEPENDENCE')
print(' ' * 65)
print(f'      = {TAU0}      vs      = {TAU0_ALT}')
print()
print(f' Δ:      |Δ | = {_fe(abs(omega_plus - op_alt))}')
print(f'      |Δ | = {_fe(abs(omega_minus - om_alt))}')
print()
print(f' ΔHat:   |Δ | = {_fe(abs(eta_plus - ep_alt))}')
print(f'      |Δ | = {_fe(abs(eta_minus - em_alt))}')

```

BASEPOINT INDEPENDENCE

$$= (0.3 + 1.2j) \quad \text{vs} \quad = (-0.2 + 0.9j)$$

$$\Delta: \quad \begin{aligned} |\Delta| &= 2.644862\text{e-}38 \\ |\Delta| &= 1.836710\text{e-}40 \end{aligned}$$

$$\Delta\text{Hat}: \quad \begin{aligned} |\Delta| &= 1.249927\text{e-}32 \\ |\Delta| &= 8.315162\text{e-}34 \end{aligned}$$

1.7 7. Structure of the analytic formula

For a **holomorphic** cusp form $f \in S_k$, the contour limit $\tau_0 \rightarrow i\infty$ gives $C_T \rightarrow 0$ (cuspidality) and $C'_S \rightarrow C_S$, yielding the classical identity:

$$[C'_S(f)]_{X^{n-j}Y^j} = (2\pi)^{n-j}(-i)^j \binom{n}{j} j! L(f, j+1),$$

where $L(f, s) = \sum_{m \geq 1} a_f(m)/m^s$ is the standard L -function, analytically continued. Dividing by $[P_S^\pm]_{X^{n-j}Y^j}$ gives $\omega^\pm$ as **explicit rational multiples of a single critical L -value**.

For **weakly holomorphic** $f \in S_k^!$, both C_S and $P|_S - P$ individually diverge as $\tau_0 \rightarrow i\infty$ (the q^{-1} mode produces $e^{2\pi\text{Im}(\tau)}$ growth), but $C'_S = C_S - (P|_S - P)$ converges: this finite limit is the **DR-regularised period integral**, which equals $\eta^\pm$ and which BFK's non-holomorphic scheme cannot reproduce.

Below we verify the L -value formula for Δ by computing $L(\Delta, j+1)$ via the BFK incomplete-gamma sum and checking $\omega^- = (2\pi)^{n-j}(-i)^j \binom{n}{j} j! L(\Delta, j+1)/[P_S^-]_j$ for each odd j .

```
[9]: from mpmath import gamma as mpgamma, gammainc

n_deg = 10 # k - 2 = 10

def L_delta_bfk(s, N=200):
    """
    L(Δ, s) via BFK incomplete-gamma formula (split-point t0=1):
    L*(Δ, s) = Σ_{m=1}^N [Γ(s, 2m)/(2m)^s + Γ(12-s, 2m)/(2m)^{12-s}]
    L(Δ, s) = (2)^s/Γ(s) · L*(Δ, s)
    """
    two_pi = 2 * pi
    L_star = mpc(0)
    for mm in range(1, N + 1):
        c = Delta_series.get(mm)
        if c is None: continue
        c = mpf(str(c))
        x = two_pi * mm
        L_star += c * (gammainc(s, x) / x**s + gammainc(12 - s, x) / x**(12 - s))
    return two_pi**s / mpgamma(s) * L_star
```

```

P_plus, P_minus = brown_basis_k12()
two_pi = 2 * pi

print('Analytic-formula verification for Δ (holomorphic case)')
print(' ' * 65)
print(f'      = {omega_minus}')
print()
print(' j      L(Δ,j+1) BFK                      recovered from          |diff|')
print(' ' + ' ' * 80)

for j in [1, 3, 5, 7, 9]:
    s = j + 1
    Lv = L_delta_bfk(s, N=150)
    # formula: [C'_S]_j = (2)^{n-j} · (-i)^j · C(n, j) · j! · L(Δ, s)
    # = [C'_S]_j / P_minus[j]      L(Δ, s) = · P_minus[j] / coeff
    jfact = mpf(1)
    for m in range(1, j + 1): jfact *= m
    coeff = two_pi**(n_deg - j) * mpc(0, -1)**j * mpf(comb(n_deg, j)) * ␣
    ↪jfact
    recovered = omega_minus * P_minus[j] / coeff
    diff = abs(Lv - recovered)
    print(f' j={j} L(Δ,{s}) = {Lv}')
    print(f'      recovered = {recovered}')
    print(f'      |diff|      = {_fe(diff)}')
    print()

```

Analytic-formula verification for Δ (holomorphic case)

= (3.67341984631964846240230167881951774318332986e-40 -
5585015.37931040186687713926379627512963503343j)

j	L(Δ,j+1) BFK	recovered from	diff
j=1	L(Δ,2) = (0.146374542091265989413000913274996215907067384 + 0.0j)	recovered = (0.146374542091265989413000913274996215907067384 + 9.62746047049199018965838112845484875945028347e-48j)	diff = 2.391844e-47
j=3	L(Δ,4) = (0.501617647510902215168742991995286352268053511 + 0.0j)	recovered = (0.501617647510902215168742991995286352268053511 + 3.2992786885722768604731946600569352018087311e-47j)	diff = 9.358944e-47
j=5	L(Δ,6) = (0.792122838646030569355944890486735838151910795 + 0.0j)	recovered = (0.792122838646030569355944890486735838151910794 + 5.21001207442452337527641510151834336782731146e-47j)	diff = 1.827464e-46

```

j=7 L( $\Delta$ ,8) = (0.930707030298126094202932748710858646123879737 + 0.0j)
      recovered = (0.930707030298126094202932748710858646123879737 +
6.12151882136535415833774735915774287646928568e-47j)
      |diff|      = 1.855509e-46

j=9 L( $\Delta$ ,10) = (0.979809088251220515857621009051378771804203405 + 0.0j)
      recovered = (0.979809088251220515857621009051378771804203405 +
6.44447670407454322102404602272435772040945621e-47j)
      |diff|      = 1.087363e-46

```

```

[10]: print('=' * 65)
      print('SUMMARY')
      print('=' * 65)
      print()
      print( $\Delta$  (holomorphic cusp form):')
      print(f'      = {nstr(omega_plus.real, 22)}      (|imag| = {_fe(abs(omega_plus.
↪imag))})')
      print(f'      /i = {nstr((omega_minus/mpc(0,1)).real, 22)}      (|real| =_
↪{_fe(abs((omega_minus/mpc(0,1)).imag))})')
      print()
      print( $\Delta$ Hat (weakly holomorphic,  $q^{-1}$  cusp pole):')
      print(f'      = {nstr(eta_plus.real, 22)}      (|imag| = {_fe(abs(eta_plus.
↪imag))})')
      print(f'      /i = {nstr((eta_minus/mpc(0,1)).real, 22)}      (|real| =_
↪{_fe(abs((eta_minus/mpc(0,1)).imag))})')
      print()
      print('Relative errors vs Brown / reference notebook:')
      print(f'       $\Delta$ :      {_fe(rel_plus)},      {_fe(rel_minus)}')
      print(f'       $\Delta$ Hat:  {_fe(rel_eplus)},      {_fe(rel_eminus)}')
      print()
      print('Basepoint shifts (  $\rightarrow$  _alt):')
      print(f'       $\Delta$ :      | $\Delta$  | {_fe(abs(omega_plus-op_alt))}, | $\Delta$  |_
↪{_fe(abs(omega_minus-om_alt))})')
      print(f'       $\Delta$ Hat: | $\Delta$  | {_fe(abs(eta_plus-ep_alt))}, | $\Delta$  |_
↪{_fe(abs(eta_minus-em_alt))})')

```

```

=====
SUMMARY
=====

```

```

 $\Delta$  (holomorphic cusp form):
      = -68916772.80959519475431      (|imag| = 8.816208e-39)
      /i = -5585015.379310401866877      (|real| = 3.673420e-40)

```

```

 $\Delta$ Hat (weakly holomorphic,  $q^{-1}$  cusp pole):
      = 127202100647.1770947773      (|imag| = 1.232595e-32)
      /i = 10276732343.64913275082      (|real| = 8.666685e-34)

```

Relative errors vs Brown / reference notebook:

Δ : 4.499988e-21, 1.381183e-20

ΔHat : 8.551348e-32, 8.961106e-32

Basepoint shifts ($\rightarrow$ _alt):

Δ : $|\Delta|$ 2.644862e-38, $|\Delta|$ 1.836710e-40

ΔHat : $|\Delta|$ 1.249927e-32, $|\Delta|$ 8.315162e-34